IDOR — Interview Prep Notes

1. Definition (logical flow)

- Insecure Direct Object Reference
- App uses user-supplied identifier → accesses resource → no ownership/authorization check
- Root: AuthN checked, AuthZ (object-level) not checked
- Taint model: source (ID param) → sink (data access) → missing sanitizer (ownership check)
- Category: Broken Access Control, not injection

2. OWASP / CWE

- OWASP Top 10 2021: **A01:2021 – Broken Access Control**
- OWASP API Security Top 10: **API1:2023 – Broken Object Level Authorization (BOLA)**
- CWE-639 – Authorization Bypass Through User-Controlled Key
- CWE-284 – Improper Access Control (parent)
- CWE-863 – Incorrect Authorization (if wrong check type used)

3. Variants

- Horizontal (same-privilege cross-user)
- Vertical (privilege escalation via object ref)
- Read-based / Write-based / Delete-based
- Indirect/hidden ref (body, header, cookie, JWT claim)
- Bulk/batch IDOR (array of IDs)
- Static file/storage IDOR (S3, CDN paths)
- Parameter pollution / alternate ID (legacy_id vs uuid)
- GraphQL resolver-level IDOR
- Encoded/encrypted reference IDOR (reversible encoding ≠ fix)
- Second-order/indirect (ID sourced from earlier response, reused elsewhere)

4. Attack Surface vs Entry Point

Surface (full inventory):

- URL path segments, query params, body (JSON/form), headers, cookies
- JWT claims, WebSocket messages, GraphQL args
- Storage paths (S3/CDN), JS bundles/mobile app hardcoded calls
- Export/download/print endpoints, password reset/unsubscribe links

- Second-order refs (ID from prior response reused later)

Entry point: the specific param/request actually tested (subset of surface)

- Surface = recon/inventory phase; Entry point = triage/test phase

5. Testing Workflow (condensed, tri-tool)

Phase 0 – Identity setup

- 2-3 accounts (A, B, low-priv C), unique marker string per object
- Burp: Session Handling Rules; ZAP: Context→Users

Phase 1 – Mapping

- Verify: full inventory of ID-bearing params
- Burp: Site Map manual crawl; ZAP: Spider + AJAX Spider
- Nuclei: N/A (consumes inventory, doesn't build it)

Phase 2 – Baseline

- Verify: normal/expected response for own object
- Burp: Repeater tab (keep as ref); ZAP: Request Editor saved response

Phase 3 – Differential testing

- Verify: cross-session access (3a horizontal, 3b vertical, 3c unauth, 3d method swap, 3e sibling endpoints, 3f second-order)
- Burp: Repeater tab groups, change request method, manual header strip
- ZAP: Request Editor → "Resend as User" dropdown, "Unauthenticated User" option
- ZAP unique: Access Control Testing add-on (auto User×URL matrix)
- Nuclei: weak fit (needs -var session tokens hardcoded); good for unauth-only checks

Phase 4 – ID manipulation/enumeration

- Verify: sequential/encoded/polluted ID variants
- Burp: Intruder (Sniper, Numbers payload, Grep-Match marker, Grep-Extract)
- ZAP: Fuzzer (Numberzz payload, Match/Filter rule)
- Nuclei: **best fit** — batch/scaled sweep of confirmed pattern via YAML template + payloads

Phase 5 – Context-specific

- GraphQL introspection/resolver test, multi-tenant cross-org test, storage direct-access test, mobile API separate proxy

Phase 6 – Confirm

- Marker string match, status+body check, write/delete state re-verify, cache header check, public/shared ruled out
- Burp: Comparer; ZAP: Compare Requests/Responses

Payloads/test cases:

- Own ID (baseline), swapped ID (horizontal/vertical), sequential ± range, encoded variants (hex/base64/leading-zero), null/0/-1/wildcard, duplicate param (pollution), method swap (GET→PUT/PATCH/DELETE)
- Why: cover missing-check, weak-check, and inconsistent-check code shapes

6. Evidence to Collect

- Two-session request/response pairs (A→B and baseline A→A)
- Unique marker string match in cross-session response
- Full status code + full response body (not just description)
- Proof object belongs to victim (from victim's own session)
- Before/after state for write/delete (independent re-fetch)
- Reproducibility (repeat 2-3x), small controlled enumeration batch for scale proof
- Proof NOT public (unauth request fails) and NOT legit shared (different tenants)

7. Good PoC Structure

1. Baseline request (User A → own object)
2. Attack request (User A → User B's object ID) + full response showing foreign data/marker
3. Reverse direction (User B → User A's object) — bidirectional proof
4. Unauth attempt (rules out "intentionally public")
5. For write/delete: pre-state, attack request, post-state (independent verification)
6. Scale proof: small enumeration sample (not full dump)
7. Impact statement tied to shown data

8. Attack Chains

- Simple horizontal → mass enumeration → bulk data exfil
- IDOR (write, email field) → password reset → ATO
- IDOR (write, role field) → privilege escalation → admin access
- Second-order IDOR → cross-service lateral movement
- Sequential ID + unauth + no rate limit → full-table breach (compliance event)

9. False Positives

- Intentionally public/shared object (no access control by design)
- Soft-deleted object returns 200 w/ empty body (status code lies)
- Demo/seed/sample data shared across all accounts
- CDN/cache serving stale response regardless of session

- Own test harness bug (wrong/expired session token)
- WAF/rate-limit response misread as auth bypass
- Legit B2B/shared-tenant access (teammates, same org)
- Object ID cosmetic/non-authoritative (slug, not real access key)
- Write returns 200 but no-op (idempotent/validation-blocked, no real state change)
- GraphQL partial null fields = correct field-level auth, not object-level bypass
- Staging/lower-env with relaxed auth (out of scope)

10. Prerequisites for Exploitability

- Valid low-priv account (or none, if fully unauth-reachable)
- Knowledge of ID/reference pattern (from own objects, JS, mobile app, error msgs)
- Network reachability to endpoint
- No knowledge of other users needed (key differentiator vs other vulns)
- Low skill/tooling bar: browser devtools/Burp/curl sufficient
- Lower bar further: sequential IDs, leaked IDs (Referer/logs/activity feed), no rate limiting

11. What Attacker Achieves

- Read: PII, financial data, documents, private messages, leaked tokens/reset links, business intel scraping
- Write: unauthorized modification, privesc via role field, ATO via email overwrite, data integrity corruption
- Delete: data loss, DoS via mass delete, audit trail destruction
- Compounding: full DB-equivalent harvest via enumeration, chaining into SSRF/session hijack, business logic abuse (coupons/inventory)

12. CIA Impact Table

Variant	Confidentiality	Integrity	Availability	Example Vector
Read-only, same-tenant	High	None	None	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N (~6.5)
Read-only, unauth, enumerable	High	None	Low	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N (~7.5)
Write, privesc, crosses boundary	High	High	None	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N (~9.6)
Write+Delete, unauth, mass-enumerable	High	High	High	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H (10.0)

13. CVSS Severity/Vector

- CWE-639 under CWE-284
- AV: almost always N
- AC: L (H only if genuine enumeration barrier)
- PR: N (unauth) / L (most common) / H (rare)
- UI: N (no victim action needed — key differentiator vs CSRF)
- S: **U** if same authz boundary; **C** if crosses tenant/privilege boundary (major score jump — always check)
- C/I/A: per variant (see table above)
- Temporal: Exploit maturity usually High (trivial to weaponize), Report Confidence High w/ marker proof
- Environmental: bump CR/IR/AR per data class regulated (PCI/HIPAA/GDPR)

14. Business Impact (Financial App Context)

- Regulatory: PCI-DSS (cardholder data), GDPR/local data protection (PII) — breach notification triggers
- Direct fraud: IDOR on transaction/account endpoints → unauthorized fund visibility/transfer
- Mass account enumeration → systemic breach, not single-account issue (regulatory reporting framing)
- Trust/reputation: customer-visible leak ("I can see others' balances") = fast escalation, PR risk
- Chain to ATO on financial accounts = critical severity regardless of base CVSS
- Compounding: audit trail tampering (write/delete on transaction logs) = compliance/audit integrity failure

15. Indicators/Signals (Detection)

- 200 OK on requests using foreign/swapped IDs where 403/404 expected
- Response schema/length matches legit-owner response (not error template)
- Sequential ID access patterns in logs (rapid incrementing requests)
- High request volume to single endpoint pattern with varying ID, low variance in other params
- Unique marker/foreign PII appearing in another user's session response
- Inconsistent response behavior across sibling endpoints (view protected, export not)

16. Root Cause (Code Level)

```
# Vulnerable
  object = db.fetch(id)      # id from request, no ownership predicate before use
  return object
```

- Missing ownership predicate entirely (authN present, authZ absent)
- Check exists but wrong scope (role check, not identity/object check)
- Check exists in query filter but bypassable (alt code path skips filter, e.g. internal header override)
- Check at UI/controller layer only, not enforced server-side/data layer
- Check present on primary endpoint, missing on sibling (duplication, not centralized)
- Type/scope confusion in comparison (str vs int ID, silent coercion)
- Comparison uses two attacker-controlled values instead of session-derived identity

17. Remediation / Secure Coding

```
# Fixed
  if not authorize(current_user, id):   # or query-scoped fetch
      abort(403)
  object = db.fetch(id)
  return object
```

- Derive identity from session/token server-side, never from request body/param
- Object-level check immediately after fetch, before return/mutate
- Prefer query-level scoping (`filter(id=id, owner=current_user.id)`) → returns 404 not 403 (avoids existence leak)
- Centralize authorization — shared `get_authorized_object()` helper, not per-endpoint reimplementation
- Policy/ABAC library at scale (OPA, Casbin, Oso) vs hand-rolled ifs
- Deny-by-default, guard clause first, not afterthought

- Negative-case test required per endpoint (CI-enforced regression)

18. Compensating Controls (if root cause fix delayed)

- WAF/API gateway rule blocking sequential ID enumeration patterns
- Rate limiting per user/session on object-fetch endpoints
- Enhanced logging/alerting on cross-account access patterns (anomaly detection)
- Switch sequential IDs → UUIDs (reduces enumeration ease, does NOT fix root cause — obscurity only)
- Temporary manual monitoring/audit of endpoint access logs
- Feature flag/disable non-critical endpoint until fixed
- Network-level restriction (internal-only) if endpoint doesn't need public exposure

19. Repeated Occurrence → SDLC Signal

- Indicates systemic gap, not isolated dev mistake
- Points to: missing secure-by-default framework pattern, no shared auth library, no negative-test culture
- Missing/ineffective SAST coverage for access-control logic flaws
- Security requirements not in design/threat-modeling phase (access control treated as afterthought)
- Training gap across teams (same mistake, multiple codebases = knowledge gap not one-off)
- Should shift from "bug list" reporting to systemic control failure reporting (compounding risk framing)

20. Upstream Improvements

- Custom SAST rules: Semgrep taint-mode (source: request param → sink: db fetch → sanitizer: ownership check)
- CodeQL for cross-function/cross-file taint tracking (catches decorator/middleware-based checks)
- Shared internal library/decorator enforcing centralized ownership check — sanctioned only path to fetch owned objects
- SAST rule simplified to: flag any direct model-fetch call NOT going through shared helper
- Rollout: WARNING mode first (tune FP rate) → promote to BLOCKING
- Baseline scan on existing repos first, separate backlog from new-code gate
- PR template checklist: negative-case test required for object-owning endpoints
- Design/threat-model checklist item: "does this endpoint enforce object-level authz"
- Track IDOR-findings-per-repo/team metric over time (validates rule effectiveness / spots training gap)

21. Sample Triage Response

```
Status: Confirmed - Valid
  Severity: High (CVSS 7.5 - AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N)
  Summary: /api/users/{id}/profile lacks object-level authorization check.
  Authenticated user can retrieve any other user's profile data by
  modifying the numeric user_id path parameter.
  Evidence: Cross-session request/response provided (Account A retrieving
  Account B's profile, marker string TESTMARKER_B confirmed in response).
  Baseline and unauthenticated tests confirm this is not intentional
  public data.
  Impact: PII disclosure across user base; sequential IDs allow mass
  enumeration (batch of 20 IDs tested, all returned foreign profile data).
  Next steps: Routed to engineering for object-level authz fix;
  retest required before closure.
```

22. Sample Developer Remediation Note

```
Issue: get_profile() (routes/users.py, line 42) fetches UserProfile by
  user_id from URL path with no ownership check — any authenticated user
  can access any profile.

  Fix: Add ownership check before fetch, or scope query directly:

      if current_user.id != user_id:
          abort(403)
      profile = db.session.get(UserProfile, user_id)

  Preferred (query-scoped, avoids existence leak, returns 404 not 403):

      profile = db.session.query(UserProfile).filter_by(
          id=user_id, owner_id=current_user.id).first()
      if profile is None:
          abort(404)

  Apply same pattern to sibling endpoints: /profile/export, /profile/download
  — check each for the same missing predicate.

  Add regression test: assert 403/404 when User A requests User B's profile.

  Reference: shared helper get_authorized_object() in auth/helpers.py —
  use this instead of direct db.session.get() going forward.
```

23. Real-World Examples

- **Facebook (2015)** — IDOR in ads API allowed access to any user's private photos via photo ID manipulation (bug bounty, Facebook Bug Bounty program)
- **USPS (2018)** — IDOR in Informed Visibility API exposed data of ~60M users via account ID enumeration, no auth needed
- **Venmo** — repeated IDOR/BOLA-class findings on transaction visibility, widely cited in API security discussions
- General class: CWE-639 has broad CVE representation across CMS platforms, e-commerce APIs, healthcare portals (HIPAA-relevant breach cases)
- Bug bounty pattern: most common BOLA reports on HackerOne/Bugcrowd involve sequential numeric IDs on REST APIs and GraphQL resolvers missing per-field authz
- *(Verify specific CVE numbers/current details before citing in interview — cite pattern/class confidently, verify exact figures if asked for precision)*